Student Registry

25 Hands-On Tasks: Local Stage

Spring Boot · Hibernate/JPA · PostgreSQL · Kafka · Testing · Docker

From running the app to production patterns, one task at a time

How to use this workbook

You already understand how the Student Registry works on paper. This workbook turns that into muscle memory. Each task is a small, realistic assignment of the kind a team lead would give a new developer. The tasks run in order, start very easy, and each one builds on the last.

- **Read "Why this matters" first.** It explains the concept in plain words before you touch anything.
- **Try the task without the hints.** Struggling for 10–15 minutes is how the learning happens. Use the hints only after that.
- **You are done when every box in "Check you're done" is true.** Don't skip the checks: they are the proof that you understood, not only that you typed.
- **Answer "Think about it" out loud or in a notes file.** These are typical interview questions.
- **Commit after every task** (see "Before you start"). If you break something, you can always go back.

The four levels

Level	Tasks	What you do	Code changes?
1 · Explore	1–6	Run the system, call every endpoint, read the SQL, look inside PostgreSQL and Kafka	None
2 · Small changes	7–12	Configuration, new endpoints, new fields, queries, timestamps, more events	Small
3 · Intermediate	13–19	Pagination, DTOs, error handling, profiles, health checks, unit and web tests, Kafka scaling	Medium
4 · Advanced	20–25	Relationships, Flyway migrations, Transactional Outbox, Dead Letter Topic, Docker, integration tests	Large

A realistic plan for your time

All 25 tasks together take roughly **20 hours**, more than one day. That is fine. Suggested split:

- **Today:** Levels 1 and 2 (Tasks 1–12, about 6–7 hours). If you still have energy, continue into Level 3.
- **Before you start on AWS:** do at least **Task 7** (environment variables), **Task 16** (health check) and **Task 24** (Docker Compose). AWS builds directly on those three.

- **Tasks 20–25** are senior-level topics. Do them after AWS if you need to; they will still be here.

Before you start (10 minutes)

Put the project under Git so you can commit after each task and undo mistakes. Open PowerShell in the `student-registry` folder:

```
git init
Set-Content .gitignore "target/`n.idea/`n*.iml`n.env"
git add -A
git commit -m "Baseline before hands-on tasks"
```

After each task:

```
git add -A
git commit -m "Task 07: externalized configuration"
```

In PowerShell, a long command continues on the next line with a backtick at the end of the line; in Git Bash it is a backslash. The code blocks in this workbook use the right one for the shell named in the task.

Useful rescue commands: `git status` (what changed), `git diff` (exact changes), `git restore .` (throw away uncommitted changes), `git log --oneline` (history).

Command cheat sheet

What	Command (PowerShell, inside student-registry)
Start Kafka	`docker compose up -d` then `docker compose ps` (wait for healthy)
Start app	`mvn spring-boot:run` (stop with Ctrl + C)
Stop Kafka	`docker compose down`
psql shell	`& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U student -h localhost -d studentdb`
Kafka tool	`docker exec -it kafka /opt/kafka/bin/<tool>.sh --bootstrap-server localhost:9092 ...`
Run tests	`mvn test`

Tip: PowerShell 5.1 has two traps. `Invoke-RestMethod` throws an exception on 4xx/5xx responses, and `curl.exe` in PowerShell strips the double quotes out of JSON bodies. When you want to see error responses, use **curl in Git Bash** (`curl -i ...`) or **Postman**.

Progress tracker

#	Task	Level	Time	Done
1	Start everything and break it on purpose	1	20 min	
2	A smoke-test script for every endpoint	1	30 min	
3	Map every error path	1	30 min	
4	Watch Hibernate write SQL	1	30 min	
5	Look inside PostgreSQL	1	30 min	
6	Look inside Kafka: offsets and lag	1	40 min	
7	Externalized configuration and env vars	2	30 min	
8	Your first endpoints: count and stats	2	30 min	
9	Add a new field: phone	2	40 min	
10	Search with derived queries	2	40 min	

#	Task	Level	Time	Done
11	Automatic audit timestamps	2	30 min	
12	Publish UPDATED and DELETED events	2	40 min	
13	Pagination and sorting	3	40 min	
14	DTOs: stop exposing the entity	3	60 min	
15	Global error handling	3	60 min	
16	Profiles, logging and Actuator health	3	45 min	
17	Unit tests with JUnit 5 and Mockito	3	60 min	
18	Web-layer tests with MockMvc	3	45 min	
19	Consumer groups and partitions	3	60 min	
20	Relationships: Department, lazy loading, N+1	4	90 min	
21	Database migrations with Flyway	4	75 min	
22	Fix the dual-write problem: Outbox	4	90 min	
23	Retries and a Dead Letter Topic	4	60 min	
24	Stage 2: everything in Docker Compose	4	90 min	
25	Capstone: integration test with Testcontainers	4	90 min	

TASK 01 **LEVEL 1 · EXPLORE** Time: ~20 min

Start everything, and break it on purpose

You will learn: the 3 running parts, reading startup logs, how the app reacts when a dependency is down

WHY THIS MATTERS

The app depends on three separate things: **PostgreSQL** (a Windows service), **Kafka** (a Docker container) and the **Spring Boot app** itself (your terminal). Starting and stopping them should feel automatic. Just as important, you should recognise from the logs which part is broken. Most real-world debugging begins with "which dependency is down?"

PROBLEM STATEMENT

You've just joined the team. Your lead says: "Get it running on your machine, prove it works, and tell me what happens if Kafka goes down."

WHAT YOU NEED TO DO

1. Start Docker Desktop and wait for "Engine running". Run `docker compose up -d`, then repeat `docker compose ps` until the kafka STATUS says **healthy**.
2. Check that PostgreSQL is running: `Get-Service postgresql*` should show **Running**.
3. Start the app with `mvn spring-boot:run`. In the log, find these three lines and write them down: (a) the Tomcat line with port 8080, (b) the Hibernate/HikariPool line showing the DB connection, (c) the Kafka line containing **partitions assigned**.
4. Open http://localhost:8080/api/students in your browser. You should see `[]` or a JSON list.
5. Break it: stop the app (Ctrl + C), run `docker compose stop kafka`, then start the app again. Does it start? What does it keep printing?
6. Fix it while the app keeps running: `docker compose start kafka`. Watch the app recover on its own.

HINTS (TRY WITHOUT THEM FIRST)

You can watch Kafka's own log with `docker compose logs -f kafka`.

The repeating warning you should see is like: `Connection to node -1 (localhost/127.0.0.1:29092) could not be established`.

CHECK YOU'RE DONE

- ☐ The browser shows a JSON response from /api/students.
- ☐ You can point to the log line that proves each of the 3 parts is connected.
- ☐ You saw the Kafka connection warning, and saw it stop once Kafka was back.

BONUS CHALLENGE

Open PowerShell **as Administrator** and run `Stop-Service postgresql-x64-18`, then try to start the app. Compare this with the Kafka-down case. Restart the service afterwards with `Start-Service`.

THINK ABOUT IT

Why does the app start without Kafka but refuse to start without PostgreSQL? (Hint: the database connection pool and Hibernate check the database during startup. Kafka clients connect lazily and keep retrying.)

TASK 02 **LEVEL 1 · EXPLORE** Time: ~30 min

A smoke-test script for every endpoint

You will learn: REST verbs, HTTP status codes, PowerShell HTTP calls, idempotent scripts

WHY THIS MATTERS

REST maps HTTP verbs to actions: POST = create, GET = read, PUT = replace, DELETE = remove. The **status code** is part of the contract. 201 means "created", 204 means "done, no body". A **smoke test** is a quick script that calls every endpoint once to prove the system basically works. You will re-run this script after almost every later task.

PROBLEM STATEMENT

QA wants a script they can run any time to check that all 5 endpoints work. It should print the status code of every call, and it must be safe to run many times in a row.

WHAT YOU NEED TO DO

1. Create the folder `scripts` and the file `scripts/smoke-test.ps1`.
2. The script must: create 3 students (different courses), list all, get one by id, update one student's course, delete one, then list all again.
3. Print the HTTP status code of every call. Use `Invoke-WebRequest`, which gives you `.StatusCode`, instead of `Invoke-RestMethod`, which gives you only the body.
4. Run it twice. The second run fails with 409. Why? Fix the script so it can run any number of times.

HINTS (TRY WITHOUT THEM FIRST)

```
$base = "http://localhost:8080/api/students"
$stamp = Get-Date -Format "HHmmssfff"          # makes every email unique
$body = @{ name = "Asha"; email = "asha$stamp@example.com"; course = "CS" } | ConvertTo-Json
$r = Invoke-WebRequest -Method Post -Uri $base -ContentType "application/json" `
                       -Body $body -UseBasicParsing
"POST -> $($r.StatusCode)"
$id = ($r.Content | ConvertFrom-Json).id
```

Run it with: `powershell -ExecutionPolicy Bypass -File scripts\smoke-test.ps1`

CHECK YOU'RE DONE

- ☐ The output shows 201, 201, 201, 200, 200, 200, 204, 200 (or similar, in order).
- ☐ Running the script 3 times in a row never gives a 409.
- ☐ You noticed that ids keep increasing and are never reused after a delete, and you can explain why (the IDENTITY sequence in PostgreSQL only moves forward).

FILES YOU'LL TOUCH

- scripts/smoke-test.ps1 (new)

BONUS CHALLENGE

Make the script fail loudly: if any status code is not what you expect, print it in red (`Write-Host ... -ForegroundColor Red`) and `exit 1`.

THINK ABOUT IT

Why does DELETE return 204 and not 200? Should DELETE of an already-deleted id return 404 or 204? (Look up "idempotent HTTP methods".)

TASK 03 **LEVEL 1 · EXPLORE** Time: ~30 min

Map every error path

You will learn: Bean Validation (@Valid), ResponseStatusException, 400 / 404 / 405 / 409 / 415, which layer rejects what

WHY THIS MATTERS

People judge an API by how it fails. In this project, errors come from different **layers**: Bean Validation (`@Valid` + `@NotBlank`/`@Email`) rejects bad input with **400**. The service rejects business-rule violations with `ResponseStatusException` (**404**, **409**). Spring MVC itself rejects wrong methods or content types (**405**, **415**). If you know which layer said no, you know where to look in the code.

PROBLEM STATEMENT

Support complains that errors are "vague". Before anyone fixes this (that's Task 15), document exactly how the API behaves today.

WHAT YOU NEED TO DO

1. Create `docs/error-catalog.md` with a table: **Request | Status | Response body (short) | Layer that rejected it | Class/line in code**.
2. Fill one row for each case: missing name · name of only spaces `" "` · invalid email · missing email · duplicate email on POST · GET id 999 · PUT id 999 · DELETE id 999 · PUT that changes a student's email to another student's email · broken JSON `{"name": }` · POST containing `"id": 500` · POST with `Content-Type: text/plain` · PATCH request.
3. Use **Git Bash** with `curl -i` so you see the status line and the headers.

HINTS (TRY WITHOUT THEM FIRST)

```
curl -i -X POST http://localhost:8080/api/students \
  -H "Content-Type: application/json" \
  -d '{"email":"b@example.com"}'
```

For validation errors, look at the **app console**. A WARN line `Resolved [...MethodArgumentNotValidException...]` names the exact field, even though the HTTP response does not.

The `"id": 500` case: look at `student.setId(null)` in `StudentService.create`.

CHECK YOU'RE DONE

- ☐ All 13 rows are filled in.
- ☐ You can explain why `"id": 500` is ignored and the database chooses the id.
- ☐ You noticed that the 400 response does **not** say which field was wrong. Note this down for Task 15.

FILES YOU'LL TOUCH

- docs/error-catalog.md (new)

THINK ABOUT IT

`@NotBlank` exists in Java **and** `NOT NULL` exists in the database. Why have both? Which one protects you if someone inserts data with a SQL tool?

TASK 04 **LEVEL 1 · EXPLORE** Time: ~30 min

Watch Hibernate write SQL

You will learn: generated SQL, dirty checking, persistence context, SQL logging options

WHY THIS MATTERS

Hibernate writes SQL for you. That is convenient, but developers who never read that SQL eventually ship slow or wrong queries. With `show-sql: true` every statement is printed. Reading it teaches you what JPA really does, especially **dirty checking**: Hibernate compares an entity with its original state when the transaction commits and runs an UPDATE only if something changed.

PROBLEM STATEMENT

A teammate insists: "PUT always runs an UPDATE, even when nothing changed." Prove or disprove it with evidence from the logs.

WHAT YOU NEED TO DO

1. For each of the 5 endpoints, write the exact SQL statements printed, in order, into `docs/sql-trace.md`.
2. Explain why POST runs a SELECT before the INSERT, and why DELETE runs a SELECT before the DELETE.
3. Send a PUT with **exactly the same values** that are already stored. Count the statements. Then send a PUT that changes only the course. Compare the two.
4. Make the SQL more readable and show the parameter values. Add this to `application.yml` and restart:

```
spring:
  jpa:
    properties:
      hibernate:
        format_sql: true
logging:
  level:
    org.hibernate.orm.jdbc.bind: trace     # prints the value bound to every ?
```

CHECK YOU'RE DONE

- ☐ `docs/sql-trace.md` covers all 5 endpoints.
- ☐ A PUT with unchanged values produced **no UPDATE**. You've disproved your teammate's claim.
- ☐ A PUT that changed only the course still updated **all** columns. You noticed this.
- ☐ You can explain "dirty checking" in one sentence.

BONUS CHALLENGE

Put `@DynamicUpdate` (org.hibernate.annotations) on `Student` and compare the UPDATE statement again. Why isn't this the default? (Hint: Hibernate can cache and reuse a fixed UPDATE statement.)

THINK ABOUT IT

`StudentService.update()` never calls `save()`. So who runs the UPDATE, and at what exact moment?

TASK 05 **LEVEL 1 · EXPLORE** Time: ~30 min

Look inside PostgreSQL

You will learn: psql basics, table structure and constraints, DB rules vs app rules

WHY THIS MATTERS

The database is the **source of truth**. Looking at it directly lets you confirm what the app really did. It also shows clearly which rules the **database** enforces (types, NOT NULL, UNIQUE) and which only your **Java code** enforces (`@NotBlank`, `@Email`).

PROBLEM STATEMENT

Your lead asks: "If another tool writes directly to our table, which of our rules still hold?" Find out by experiment.

WHAT YOU NEED TO DO

1. Open psql: `& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U student -h localhost -d studentdb` (password `student123`).
2. Run `\dt` and `\d students`. Write down every column, its type, and every constraint. Find the name of the unique constraint on email and how the id is generated.
3. Insert a row directly with SQL, then GET it through the API. The app never saw the insert, so why does it return the row?

```
INSERT INTO students (name, email, course) VALUES ('Ravi', 'ravi@example.com', 'Maths');
SELECT * FROM students ORDER BY id;
```

4. Insert a duplicate email with SQL. Read the error and compare it with the 409 the API gives.
5. Insert a row with `name = ''` (empty string). It **succeeds**. Now send a PUT for that student through the API, keeping all its values. It fails with 400. Explain why.
6. Change a row with SQL `UPDATE` and GET it via the API. Is the API showing the fresh value?
7. Delete the bad rows you created.

HINTS (TRY WITHOUT THEM FIRST)

Useful psql commands: `\x` (toggle vertical display), `\d+ students` (more detail), `\q` (quit).

CHECK YOU'RE DONE

- ☐ You can name 3 rules the **database** enforces and 2 rules that only the **application** enforces.
- ☐ You saw an empty-name row get into the DB despite `@NotBlank`.

THINK ABOUT IT

If a second application writes to this database, which validations protect your data? Where should the most important rules live?

TASK 06 **LEVEL 1 · EXPLORE** Time: ~40 min

Look inside Kafka: topics, offsets and lag

You will learn: topics and partitions, offsets, consumer groups, consumer lag, message retention

WHY THIS MATTERS

Kafka isn't a queue that deletes a message once it's read. It is an **append-only log**. Messages stay for the retention period (7 days by default). Each **consumer group** stores its own position in the log (the **offset**). **Lag** is how many messages the group has not processed yet. When you see lag go up while the app is down, then drop back to 0 when it restarts, Kafka starts to make sense.

PROBLEM STATEMENT

Ops asks: "If the app is down for an hour, do we lose the welcome-email events?" Answer them with a live demonstration, not a guess.

WHAT YOU NEED TO DO

1. List the topics, then describe `student-events` (number of partitions, leader):

```
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 `
  --describe --topic student-events
```

2. Read all messages, with their key, partition and offset (Ctrl + C to stop):

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 `
  --topic student-events --from-beginning `
  --property print.key=true --property print.partition=true --property print.offset=true
```

3. Describe the app's consumer group. Note CURRENT-OFFSET, LOG-END-OFFSET and LAG:

```
docker exec -it kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 `
  --describe --group student-registry-group
```

4. **Stop the app.** Send 2 messages by hand with the console producer (type a line, press Enter, then Ctrl + C):

```
docker exec -it kafka /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:9092 `
  --topic student-events
```

5. Describe the group again. LAG should now be **2**.
6. Start the app. It should log "Received event" for both messages. Describe the group again: LAG is back to **0**.
7. Write your answer to Ops in 2–3 sentences in `docs/kafka-notes.md`.

> **Heads-up:** The 2 plain-text messages you just typed are not JSON. Today the consumer accepts any string. In Task 19 you'll make it parse JSON, and these messages will start causing errors. That is intentional: Task 23 is about handling these "poison" messages properly.

HINTS (TRY WITHOUT THEM FIRST)

In this task you run Kafka's tools **inside** the container, so the address is `localhost:9092`, not 29092. Your notes from the guide on the two Kafka ports explain why.

CHECK YOU'RE DONE

- ☐ You saw LAG go above 0 while the app was down, and back to 0 after it restarted.
- ☐ You can explain: topic, partition, offset, consumer group, lag, retention.
- ☐ `docs/kafka-notes.md` contains your answer to Ops.

FILES YOU'LL TOUCH

- docs/kafka-notes.md (new)

BONUS CHALLENGE

Run the console consumer with `--group my-test-group --from-beginning` twice. Why does the second run print nothing?

THINK ABOUT IT

Which setting decides how long Kafka keeps messages (`retention.ms`)? What happens if the app is down for longer than that?

TASK 07 **LEVEL 2 · SMALL CHANGES** Time: ~30 min

Externalized configuration and environment variables

You will learn: ${VAR:default} placeholders, configuration precedence, relaxed binding, @Value, running 2 instances

WHY THIS MATTERS

The **same JAR** must run on your laptop, in Docker and on AWS. Only the **configuration** changes. Spring Boot reads settings from several sources in a fixed priority order: command-line arguments beat environment variables, and environment variables beat `application.yml`. Tomorrow, on AWS, you'll configure the app only through environment variables. Passwords must also never sit in Git.

PROBLEM STATEMENT

A security review flagged a database password committed in `application.yml`. Separately, the team wants to run two copies of the app side by side.

WHAT YOU NEED TO DO

1. Replace the hard-coded values with placeholders that have local defaults: `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `KAFKA_BOOTSTRAP_SERVERS`. Example: `password: ${DB_PASSWORD:student123}`.
2. Set a wrong password and start the app: `env:DB_PASSWORD="wrong"; mvn spring-boot:run`. Read the error. Then `Remove-Item Env:DB_PASSWORD` and start again.
3. Run 2 instances at the same time: a second terminal with `env:SERVER_PORT="8081"`. POST a student to port 8080. **Which instance's consumer logs the event?** Stop that instance, watch the other one log the rebalance (partitions assigned), then POST again.
4. Add a custom property `app.welcome-message: "Welcome to the registry"`. Use it in `StudentEventConsumer` with `@Value("${app.welcome-message}")`. Override it with the environment variable `APP_WELCOMEMESSAGE`.

HINTS (TRY WITHOUT THEM FIRST)

An environment variable set in PowerShell lasts only for that terminal window.

Relaxed binding: `app.welcome-message` becomes the env var `APP_WELCOMEMESSAGE` (upper case, dots become _, dashes removed).

Command-line arguments win over everything: `mvn spring-boot:run "-Dspring-boot.run.arguments=--server.port=8082"`.

CHECK YOU'RE DONE

- ☐ `application.yml` contains only placeholders (with local defaults).
- ☐ A wrong DB_PASSWORD makes startup fail, and you understood the error.
- ☐ With 2 instances in the same consumer group, only **one** received each event, and you know why (the topic has 1 partition, and a partition belongs to one consumer in a group).
- ☐ Your custom message changed when you set the env var.

FILES YOU'LL TOUCH

- src/main/resources/application.yml
- kafka/StudentEventConsumer.java

THINK ABOUT IT

`${DB_PASSWORD:student123}` still has the password in Git as the default. How would you remove it entirely? (Tomorrow's answer: AWS SSM Parameter Store or Secrets Manager.)

TASK 08 **LEVEL 2 · SMALL CHANGES** Time: ~30 min

Your first endpoints: count and stats

You will learn: controller → service → repository, inherited repository methods, JPQL @Query, constructor projections

WHY THIS MATTERS

Adding a feature end to end through all three layers is the most common day-to-day task. You'll also learn that the repository can do much more than `findAll()`: it can count and group in the **database**, which is far faster than loading every row into Java and counting there.

PROBLEM STATEMENT

The dashboard team needs (a) the total number of students and (b) the number of students per course.

WHAT YOU NEED TO DO

1. `GET /api/students/count` returns `{"count": 3}`.
2. `GET /api/students/stats/by-course` returns `[{"course":"CS","count":2}, {"course":"Maths","count":1}]`.
3. Check the SQL: you should see `count(*)` and `group by`. If you see a plain `select ... from students`, you are counting in Java. Fix that.

HINTS (TRY WITHOUT THEM FIRST)

`repository.count()` is already inherited from `JpaRepository`. Return `Map.of("count", n)` for a quick JSON object.

For grouping, use a record and a JPQL constructor expression:

```
public record CourseCount(String course, long count) { }

@Query("select new com.example.studentregistry.model.CourseCount(s.course, count(s)) "
     + "from Student s group by s.course")
List<CourseCount> countByCourse();
```

CHECK YOU'RE DONE

- ☐ Both endpoints return correct numbers. Verify them with a SQL query in psql.
- ☐ `GET /api/students/5` still works (there is no clash between `/count` and `/{id}`).
- ☐ The logged SQL uses COUNT and GROUP BY.

FILES YOU'LL TOUCH

- controller/StudentController.java
- service/StudentService.java
- repository/StudentRepository.java
- model/CourseCount.java (new)

BONUS CHALLENGE

Students without a course are grouped under `null`. Show them as "Unassigned" with `coalesce(s.course, 'Unassigned')`.

THINK ABOUT IT

Why doesn't `/count` collide with `/{id}`? What happens if a client calls `/api/students/abc`?

TASK 09 **LEVEL 2 · SMALL CHANGES** Time: ~40 min

Add a new field: phone

You will learn: schema evolution with ddl-auto, @Pattern, null vs blank in validation, finding a real bug

WHY THIS MATTERS

Changing the schema is everyday work. `ddl-auto: update` adds new columns for you, which is easy, but you'll see its limits. This task also hides a **real bug** in the existing code that only appears once you add a field. Finding it is part of the task.

PROBLEM STATEMENT

Admissions wants an optional phone number. If a phone number is given, it must be exactly 10 digits.

WHAT YOU NEED TO DO

1. Add a `phone` field to `Student` (with getter and setter) and validate it with `@Pattern(regexp = "\\d{10}", message = "...")`.
2. Restart. Find the `alter table` statement in the log and check `\d students` in psql.
3. Test: POST with a valid phone gives 201, with `"12ab"` gives 400, and with **no** phone gives 201. Why is a missing phone allowed?
4. PUT an existing student with a new phone, then GET it. **Was the phone saved?** Find out why not, and fix it.
5. Add phone to your smoke-test script.

HINTS (TRY WITHOUT THEM FIRST)

`@Pattern` treats `null` as valid. To make a field required, combine it with `@NotBlank`.

The bug is in `StudentService.update()`. Look at which fields it copies.

CHECK YOU'RE DONE

- ☐ Valid, invalid and missing phone numbers behave as described.
- ☐ PUT now saves the phone. You found and fixed the bug in `update()`.
- ☐ Existing rows have `phone = NULL` in the database.

FILES YOU'LL TOUCH

- model/Student.java
- service/StudentService.java
- scripts/smoke-test.ps1

BONUS CHALLENGE

Rename the column with `@Column(name = "mobile")` and restart. Look at the table: `ddl-auto` added a **new** column and left the old one, data and all. This is exactly why Task 21 introduces Flyway. Undo the rename afterwards.

THINK ABOUT IT

Copying fields one by one in `update()` is fragile: every new field risks the same bug. How could you design this so it can't happen? (Task 14 is part of the answer.)

TASK 10 **LEVEL 2 · SMALL CHANGES** Time: ~40 min

Search with derived queries

You will learn: derived query methods, @RequestParam, optional parameters, @Query (JPQL and native SQL)

WHY THIS MATTERS

Spring Data reads a repository method name such as `findByCourseIgnoreCase` and builds the SQL for you. You already have one example: `existsByEmail`. Learning the naming rules saves you a lot of boilerplate. You also need to know when the method names get too long and you should write the query yourself.

PROBLEM STATEMENT

The front desk needs to search students by course (exact match, case-insensitive) and by part of a name.

WHAT YOU NEED TO DO

1. `GET /api/students/search?course=computer science` returns students whose course matches, ignoring case.
2. `GET /api/students/search?name=ash` returns students whose name contains "ash", ignoring case.
3. Both parameters together combine with AND. No parameters at all should return 400 with the message "Provide course or name".
4. Read the SQL in the logs. Look for `upper(...)` and `like`.

HINTS (TRY WITHOUT THEM FIRST)

Method names: `findByCourseIgnoreCase`, `findByNameContainingIgnoreCase`, `findByNameContainingIgnoreCaseAndCourseIgnoreCase`.

`@RequestParam(required = false) String course`.

Spaces in URLs: use `?course=computer%20science`, or `curl -G --data-urlencode "course=computer science" ...`.

CHECK YOU'RE DONE

- ☐ All 4 cases behave as described.
- ☐ A search with no matches returns **200 with** `[]`, not 404, and you can explain why that is the REST convention.

FILES YOU'LL TOUCH

- controller/StudentController.java
- service/StudentService.java
- repository/StudentRepository.java

BONUS CHALLENGE

Write the name search a second time as `@Query` JPQL, then a third time as native SQL (`nativeQuery = true`). Compare the logged SQL for all three.

THINK ABOUT IT

When would you stop using derived method names and switch to `@Query` or Specifications?

TASK 11 **LEVEL 2 · SMALL CHANGES** Time: ~30 min

Automatic audit timestamps

You will learn: @CreationTimestamp / @UpdateTimestamp, updatable = false, read-only JSON fields, Instant and UTC

WHY THIS MATTERS

Almost every real table records when a row was created and when it last changed. Hibernate can fill these columns for you, so no developer can forget to. This task also connects back to dirty checking from Task 4.

PROBLEM STATEMENT

Support keeps asking: "When did this student register, and when were they last changed?"

WHAT YOU NEED TO DO

1. Add `createdAt` (set on insert, never changes) and `updatedAt` (changes on every real update) as `Instant` fields.
2. Clients must not be able to set these fields. A `"createdAt"` in the POST body must be ignored.
3. Check that a PUT with **unchanged** values does **not** change `updatedAt`. Why not? (Task 4.)
4. Check the column type in psql (`timestamp(6) with time zone`) and the JSON format (ends in `Z` = UTC).
5. Existing rows have `NULL` timestamps. Backfill them with one SQL `UPDATE` statement.

HINTS (TRY WITHOUT THEM FIRST)

```
@CreationTimestamp
@Column(updatable = false)
@JsonProperty(access = JsonProperty.Access.READ_ONLY)
private Instant createdAt;

@UpdateTimestamp
@JsonProperty(access = JsonProperty.Access.READ_ONLY)
private Instant updatedAt;
```

Alternative without Hibernate-specific annotations: JPA callbacks `@PrePersist` and `@PreUpdate`.

CHECK YOU'RE DONE

- ☐ POST returns both timestamps. PUT with a real change updates only `updatedAt`.
- ☐ Sending `createdAt` in the body has no effect.
- ☐ No row has NULL timestamps after the backfill.

FILES YOU'LL TOUCH

- model/Student.java

THINK ABOUT IT

Should timestamps come from the Java clock or the database clock (`now()`)? What goes wrong when 3 servers have slightly different clocks?

TASK 12 **LEVEL 2 · SMALL CHANGES** Time: ~40 min

Publish UPDATED and DELETED events

You will learn: event-driven design, message keys and ordering, static factory methods, a hidden transaction risk

WHY THIS MATTERS

In an event-driven system, **every** change is announced, not only creations. Other services (email, analytics, search) subscribe and react without the registry knowing they exist. Because you use the student id as the message key, all events for one student go to the same partition and arrive **in order**.

PROBLEM STATEMENT

The analytics team needs to know about every change: created, updated and deleted.

WHAT YOU NEED TO DO

1. Publish `STUDENT_UPDATED` at the end of `update()` and `STUDENT_DELETED` in `delete()`.
2. The DELETED event must still contain the student's name and email, so read them before you delete.
3. Remove the repeated `new StudentEvent(...)` code with a static factory: `StudentEvent.of(String type, Student s)`.
4. Create, update and delete one student. Watch the console consumer (Task 6 command with `print.key=true`): 3 events, in order, all with the same key.

HINTS (TRY WITHOUT THEM FIRST)

Keep the event type names in constants (or an enum) so there are no typos like `STUDENT_UPDATE`.

CHECK YOU'RE DONE

- ☐ You see 3 events for one student, in order, all with the same key.
- ☐ The DELETED event contains the name and email.
- ☐ There is no duplicated event-building code in the service.

FILES YOU'LL TOUCH

- kafka/StudentEvent.java
- service/StudentService.java

BONUS CHALLENGE

Publish `STUDENT_UPDATED` only if something actually changed. Compare the old and new values before you copy them.

THINK ABOUT IT

The event is sent **inside** the `@Transactional` method, before the commit. In `update()`, the UPDATE SQL only runs at commit time. What happens if the database rejects that UPDATE after the event was already sent? (Task 22 fixes this.)

Pagination and sorting

You will learn: Pageable, Page<T>, page-size limits, LIMIT/OFFSET SQL, count queries

WHY THIS MATTERS

`GET /api/students` returns **every** row. With 100,000 students that is slow, uses a lot of memory, and can crash the browser. Real APIs return one **page** at a time with metadata (total count, total pages). Spring Data does most of the work through the `Pageable` parameter.

PROBLEM STATEMENT

The front-end shows 10 students per page, sortable by name or by registration date.

WHAT YOU NEED TO DO

1. Add 50 test students quickly with one SQL statement:

```
INSERT INTO students (name, email, course)
SELECT 'Student ' || g,
       'student' || g || '@example.com',
       (ARRAY['CS','Maths','Physics'])[1 + g % 3]
FROM generate_series(1, 50) AS g;
```

2. `GET /api/students/paged?page=0&size=10&sort=name,asc` returns one page: the content plus totalElements, totalPages, page number and size.
3. Cap the page size: a client asking for `size=10000` gets at most 50.
4. Read the SQL: one query with `offset ... fetch first ... rows only`, and a second `count` query. Why the second one?

HINTS (TRY WITHOUT THEM FIRST)

Controller: `public Page<Student> paged(Pageable pageable)`. `repository.findAll(pageable)` already exists.

Spring Boot 3.3 logs a warning about serializing `PageImpl` directly. Fix it with `@EnableSpringDataWebSupport(pageSerializationMode = PageSerializationMode.VIA_DTO)` on the application class, or return your own `PageResponse` record.

Size cap property: `spring.data.web.pageable.max-page-size: 50`.

CHECK YOU'RE DONE

- ☐ Pages, sorting and the size cap all work.
- ☐ You saw the two SQL statements and can explain the count query.
- ☐ The old `GET /api/students` still works, so you didn't break existing clients.

FILES YOU'LL TOUCH

- controller/StudentController.java
- StudentRegistryApplication.java
- application.yml

BONUS CHALLENGE

Try `sort=doesNotExist`. What status code do you get? It should be 400. Note it down and fix it in Task 15.

THINK ABOUT IT

Offset pagination gets slow at page 10,000 because the database still walks past all the skipped rows. What is **keyset (cursor) pagination**, and when is it better?

TASK 14 **LEVEL 3 · INTERMEDIATE** Time: ~60 min

DTOs: stop exposing the entity

You will learn: request/response DTOs, Java records, mass assignment, mapping layer, API vs DB decoupling

WHY THIS MATTERS

Right now the API receives and returns the `@Entity` itself. That causes three problems. (1) Every database change becomes an API change. (2) Clients can send fields they shouldn't control (`id`, `createdAt`): this is called **mass assignment**. (3) In Task 20, lazy-loaded relations will break JSON serialization. A **DTO** (Data Transfer Object) gives the API its own shape, separate from the table. Your `StudentEvent` record is already a DTO for Kafka.

PROBLEM STATEMENT

A security review says: "The API accepts fields it shouldn't, and every DB change breaks the API contract."

WHAT YOU NEED TO DO

1. Create the package `dto` with `StudentRequest` (name, email, course, phone, plus the validation annotations) and `StudentResponse` (id, name, email, course, phone, createdAt, updatedAt).
2. The controller accepts and returns **only** DTOs. The entity never leaves the service layer.
3. Put all the mapping in one place: a `StudentMapper` class, or static methods such as `StudentResponse.from(Student)`.
4. Keep the paged endpoint working: `page.map(StudentResponse::from)`.
5. Run your smoke-test script. It is now your **regression test**: nothing visible should have changed.

HINTS (TRY WITHOUT THEM FIRST)

```
public record StudentRequest(
        @NotBlank String name,
        @NotBlank @Email String email,
        String course,
        @Pattern(regexp = "\\d{10}") String phone) { }
```

CHECK YOU'RE DONE

- ☐ `StudentController` has no import of `model.Student`.
- ☐ Sending `"id"` or `"createdAt"` in a request can no longer have any effect, by design.
- ☐ The smoke test passes unchanged.

FILES YOU'LL TOUCH

- dto/StudentRequest.java (new)
- dto/StudentResponse.java (new)
- controller/StudentController.java
- service/StudentService.java

BONUS CHALLENGE

Look up MapStruct, a library that generates mappers at compile time. When would it be worth adding?

THINK ABOUT IT

Spring Boot silently ignores unknown JSON properties (a client sends `"nmae"` instead of `"name"`). Is that good or bad? How would you make it strict?

TASK 15 **LEVEL 3 · INTERMEDIATE** Time: ~60 min

Global error handling

You will learn: @RestControllerAdvice, @ExceptionHandler, domain exceptions, consistent error format, never leak stack traces

WHY THIS MATTERS

A professional API returns **one** consistent error format that clients can rely on. `@RestControllerAdvice` lets you handle exceptions from every controller in one place. You'll also replace `ResponseStatusException` with **domain exceptions** (`StudentNotFoundException`). The service then describes *what* went wrong, and the web layer decides *which HTTP status* that becomes.

PROBLEM STATEMENT

The front-end team needs one error format, with a message for each field so they can show it next to the form input. Your error catalog from Task 3 is the list of test cases.

WHAT YOU NEED TO DO

1. Every error response must look like this:

```
{
  "timestamp": "2026-10-04T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/students",
  "fieldErrors": { "email": "must be a well-formed email address", "name": "must not be blank" }
}
```

2. Create an `ApiError` record and a `GlobalExceptionHandler` class annotated with `@RestControllerAdvice`.
3. Create `StudentNotFoundException` and `DuplicateEmailException` and use them in the service instead of `ResponseStatusException`.
4. Handle: validation errors (400 + fieldErrors) · malformed JSON (400) · not found (404) · duplicate (409) · bad sort property from Task 13 (400) · `DataIntegrityViolationException` (409) · **any other exception** (500 with a generic message; log the full stack trace on the server).
5. Re-run every row of `docs/error-catalog.md` and update the table with the new responses.

HINTS (TRY WITHOUT THEM FIRST)

`MethodArgumentNotValidException` → `ex.getBindingResult().getFieldErrors()` gives field + message.

`HttpMessageNotReadableException` = broken JSON. `PropertyReferenceException` = bad sort field.

Add an `HttpServletRequest request` parameter to a handler method to get `request.getRequestURI()`.

CHECK YOU'RE DONE

- ☐ Every error has the same JSON shape, and validation errors list every bad field.
- ☐ No response anywhere contains a Java stack trace or a class name.
- ☐ The 500 handler logs the full exception on the server.

FILES YOU'LL TOUCH

- exception/GlobalExceptionHandler.java (new)
- exception/ApiError.java (new)
- exception/StudentNotFoundException.java (new)

- exception/DuplicateEmailException.java (new)
- service/StudentService.java
- docs/error-catalog.md

BONUS CHALLENGE

Spring has a standard error format built in: `ProblemDetail` (RFC 9457). Try returning `ProblemDetail` instead of your own record. Which would you choose for a public API?

THINK ABOUT IT

Why is it dangerous to put `exception.getMessage()` from an **unexpected** exception into the response body?

TASK 16 **LEVEL 3 · INTERMEDIATE** Time: ~45 min

Profiles, logging and Actuator health

You will learn: Spring profiles, logger levels vs show-sql, Spring Boot Actuator, health / info / metrics, liveness vs readiness

WHY THIS MATTERS

In development you want detailed SQL logs. In production you want quiet logs and a **health URL** that a load balancer can call to ask "is this instance OK?". Tomorrow on AWS, the load balancer or container service will call exactly that URL. **Profiles** let one codebase carry different settings for each environment.

PROBLEM STATEMENT

Get the app ready for deployment: settings per environment, and a health endpoint that really checks the database.

WHAT YOU NEED TO DO

1. Create `application-dev.yml` and move all the noisy SQL logging there. Base `application.yml` stays quiet.
2. Replace `show-sql` (which prints straight to the console) with the logger `logging.level.org.hibernate.SQL: debug`.
3. Run with the dev profile: `mvn spring-boot:run "-Dspring-boot.run.profiles=dev"` (or `$env:SPRING_PROFILES_ACTIVE="dev"`).
4. Add the dependency `spring-boot-starter-actuator`. Expose `health,info,metrics`, and show health details in dev only.
5. `GET /actuator/health` → `UP` with a `db` component. Stop PostgreSQL (admin PowerShell: `Stop-Service postgresql-x64-18`) → `DOWN` with HTTP **503**. Start it again → `UP`.
6. `GET /actuator/metrics/http.server.requests`: how many requests has the app served?
7. Show the app version at `/actuator/info`: `info.app.version: "@project.version@"` plus `management.info.env.enabled: true`.

HINTS (TRY WITHOUT THEM FIRST)

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: always      # put this in application-dev.yml only
```

CHECK YOU'RE DONE

- ☐ Without the dev profile the console is quiet; with it you see SQL.
- ☐ Health goes UP → DOWN (503) → UP as you stop and start PostgreSQL.
- ☐ /actuator/info shows the version from pom.xml.

FILES YOU'LL TOUCH

- application.yml
- application-dev.yml (new)
- pom.xml

BONUS CHALLENGE

There is no Kafka entry in health: Spring Boot has no built-in Kafka health indicator. Write your own `HealthIndicator` that calls `AdminClient.describeCluster()` with a short timeout.

THINK ABOUT IT

Should a load balancer health check fail when **Kafka** is down? If it does, every instance gets restarted at once, and that doesn't fix Kafka. Look up "liveness vs readiness probes".

TASK 17 **LEVEL 3 · INTERMEDIATE** Time: ~60 min

Unit tests with JUnit 5 and Mockito

You will learn: JUnit 5, Mockito mocks, verify / never(), ArgumentCaptor, given-when-then, proving a test catches a bug

WHY THIS MATTERS

Tests let you change code without fear. A **unit test** checks one class in isolation. Its collaborators (repository, producer) are replaced with **mocks**, so there is no database and no Kafka and the tests run in milliseconds. The service is the best place to start because it holds the business rules.

PROBLEM STATEMENT

Your lead's new rule: "No more merges without tests for the StudentService rules."

WHAT YOU NEED TO DO

1. Add `spring-boot-starter-test` (scope `test`) to pom.xml.
2. Create `src/test/java/com/example/studentregistry/service/StudentServiceTest.java`.
3. Write at least these tests: (1) create saves and publishes `STUDENT_CREATED` with the **saved** id; (2) a duplicate email throws and **never** saves or publishes; (3) findById of a missing id throws not-found; (4) update to another student's email throws conflict; (5) update keeping the same email never calls `existsByEmail`; (6) delete of a missing id throws and never calls `delete`.
4. Run `mvn test`. Everything should pass.
5. **Prove the tests work**: delete the duplicate-email check from the service, run the tests and watch one fail. Then restore the check.

HINTS (TRY WITHOUT THEM FIRST)

```
@ExtendWith(MockitoExtension.class)
class StudentServiceTest {

    @Mock StudentRepository repository;
    @Mock StudentEventProducer producer;
    @InjectMocks StudentService service;

    @Test
    void create_rejectsDuplicateEmail() {
        // given
        when(repository.existsByEmail("a@x.com")).thenReturn(true);
        // when + then
        assertThrows(DuplicateEmailException.class, () -> service.create(/* request */));
        verify(repository, never()).save(any());
        verify(producer, never()).publish(any());
    }
}
```

To check what was published: `ArgumentCaptor<StudentEvent> captor = ArgumentCaptor.forClass(StudentEvent.class);` then `verify(producer).publish(captor.capture())`.

> **Careful:** Do **not** use `@SpringBootTest` here. It starts the whole app and needs PostgreSQL and Kafka. That's Task 25.

CHECK YOU'RE DONE

- ☐ At least 6 tests pass, and `mvn test` finishes in a few seconds.

☐ You saw a test go red when you removed a rule.

FILES YOU'LL TOUCH

- pom.xml
- src/test/java/.../service/StudentServiceTest.java (new)

THINK ABOUT IT

These tests mock the repository. What do they **not** prove? (For example: that the SQL actually works, or that the unique constraint exists.)

TASK 18 **LEVEL 3 · INTERMEDIATE** Time: ~45 min

Web-layer tests with MockMvc

You will learn: @WebMvcTest slice, @MockBean, MockMvc, jsonPath assertions, testing the HTTP contract

WHY THIS MATTERS

Service tests don't check URLs, JSON shapes, status codes, validation or your error handler. `@WebMvcTest` starts **only** the web layer (controllers, advice, JSON). There is no database and no real server, and the service is a mock. These tests protect the **contract** that the front-end depends on.

PROBLEM STATEMENT

Last week someone renamed a JSON field and broke the front-end. Add tests that would have caught it.

WHAT YOU NEED TO DO

1. Create `StudentControllerTest` with `@WebMvcTest(StudentController.class)`, `@MockBean StudentService` and `@Autowired MockMvc`.
2. Test: POST valid → 201 and `$.id` exists · POST blank name → 400 and `$.fieldErrors.name` exists · GET unknown id (the mock throws not-found) → 404 in your error format · DELETE → 204 · broken JSON → 400 · GET paged → 200 and `$.content` is an array.
3. Run `mvn test`. Both test classes run.

HINTS (TRY WITHOUT THEM FIRST)

```
mockMvc.perform(post("/api/students")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"name":"", "email":"a@x.com"}
            """))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.fieldErrors.name").exists());
```

In Spring Boot 3.3, `@MockBean` is correct. (Spring Boot 3.4 and later replace it with `@MockitoBean`.)

CHECK YOU'RE DONE

- ☐ At least 6 web tests pass.
- ☐ Rename a field in `StudentResponse` and watch a test fail. Then revert the rename.

FILES YOU'LL TOUCH

- src/test/java/.../controller/StudentControllerTest.java (new)

THINK ABOUT IT

Give one example of a bug a MockMvc test catches that a service unit test cannot, and one the other way round.

TASK 19 **LEVEL 3 · INTERMEDIATE** Time: ~60 min

Consumer groups, partitions and parallel consumers

You will learn: multiple consumer groups, partitions as the unit of parallelism, listener concurrency, key → partition, default retry behaviour

WHY THIS MATTERS

This is Kafka's scaling model in one task. Inside **one group**, each partition is read by exactly one consumer, so more partitions allow more parallel consumers. **Different groups** each get **every** message independently. That is how an email service and an audit service can both read the same topic without interfering.

PROBLEM STATEMENT

Two new requirements: (a) an Audit service must record every event, independently of email sending; (b) sending emails is slow (2 s each), so it has to scale out.

WHAT YOU NEED TO DO

1. Make `StudentEventConsumer` parse the JSON into a `StudentEvent` (inject Spring's `ObjectMapper`) and log its fields. Add `Thread.sleep(2000)` to simulate a slow email.
2. Create `AuditEventConsumer` with `groupId = "audit-group"` that logs `AUDIT: ...`. Restart. What does the audit group do with the **old** messages? (Look at `auto-offset-reset` in application.yml.)
3. Increase the topic to 3 partitions (`partitions(3)` in KafkaConfig, then restart) and set `concurrency = "3"` on the email listener. Log the partition number and thread name for each message.
4. Create 6 students quickly (loop in your script). Watch 3 threads work in parallel. Confirm that all events for one student id always land on the same partition.
5. Describe both groups with `kafka-consumer-groups.sh --describe`. How many consumers does each have, and which partitions?

> **Expect errors:** The 2 plain-text messages from Task 6 now fail JSON parsing. Read the error log: how many times is each one retried before Spring gives up and skips it? Remember the number for Task 23.

HINTS (TRY WITHOUT THEM FIRST)

```
@KafkaListener(topics = StudentEventProducer.TOPIC, groupId = "...", concurrency = "3")
```

Partition number: add a method parameter `@Header(KafkaHeaders.RECEIVED_PARTITION) int partition`.

Thread name: `Thread.currentThread().getName()`.

CHECK YOU'RE DONE

- ☐ `--describe --topic student-events` shows 3 partitions.
- ☐ The email group has 3 consumers with 1 partition each. The audit group gets every message separately.
- ☐ You can explain why the audit group replayed the history.

FILES YOU'LL TOUCH

- kafka/StudentEventConsumer.java
- kafka/AuditEventConsumer.java (new)
- kafka/KafkaConfig.java

THINK ABOUT IT

Why can't a group have more **active** consumers than the topic has partitions? And why is adding partitions to an existing topic risky for message order? (Hint: key → partition is `hash(key) % partitions`.)

TASK 20 **LEVEL 4 · ADVANCED** Time: ~90 min

Relationships: Department, lazy loading and N+1

You will learn: @ManyToOne / @JoinColumn, foreign keys, FetchType.LAZY, LazyInitializationException, the N+1 problem, @EntityGraph / join fetch

WHY THIS MATTERS

Real data is relational. JPA relationships are powerful, but they cause the two most common Hibernate problems in production: **LazyInitializationException** and the **N+1 query problem** (1 query for the list, then 1 more query for **each** row). Both are favourite interview topics. Here you'll trigger both deliberately and then fix them.

PROBLEM STATEMENT

The university is adding departments. Each student belongs to at most one department.

WHAT YOU NEED TO DO

1. Create a `Department` entity (id, unique `code` such as "CSE", name) with `POST /api/departments`, `GET /api/departments` and `GET /api/departments/{id}`.
2. Add `@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "department_id") Department department` to Student. Look at `\d students`: there should be a new **foreign key**.
3. `PUT /api/students/{id}/department/{departmentId}` assigns a department. `StudentResponse` gets the flat fields `departmentCode` and `departmentName`. Do not nest the entity.
4. `GET /api/departments/{id}/students` lists the students in a department.
5. **Trigger N+1:** create 3 departments and spread 20 students across them, then GET /api/students and count the SELECT statements. **Fix it** with `@EntityGraph(attributePaths = "department")` or a `join fetch` query, then count again.
6. Try to delete a department that still has students. You get a foreign key violation. Return 409 from your error handler.

HINTS (TRY WITHOUT THEM FIRST)

If you see `could not initialize proxy - no Session`, you touched a lazy field **after** the transaction ended. Remember `open-in-view: false`. Fix: do the entity → DTO mapping **inside** a `@Transactional(readOnly = true)` service method.

You don't need `@OneToMany` on Department. A repository method `findByDepartmentId(Long id)` is simpler.

Hibernate loads each department only once per transaction, so N+1 shows as "1 + number of distinct departments" SELECTs, not 1 + 20.

CHECK YOU'RE DONE

- ☐ The FK exists, assignment works, and the response shows department code and name.
- ☐ You saw the N+1 SELECTs and reduced them to 1 query with a join.
- ☐ Deleting a department that has students gives 409, not 500.

FILES YOU'LL TOUCH

- model/Department.java (new)
- repository/DepartmentRepository.java (new)
- controller/DepartmentController.java (new)
- service/DepartmentService.java (new)

- model/Student.java
- dto/StudentResponse.java

THINK ABOUT IT

JPA's default for `@ManyToOne` is **EAGER**. Why do experienced developers almost always change it to LAZY?

TASK 21 **LEVEL 4 · ADVANCED** Time: ~75 min

Database migrations with Flyway

You will learn: versioned migrations, flyway_schema_history, baseline an existing DB, ddl-auto: validate, never edit an applied migration

WHY THIS MATTERS

`ddl-auto: update` can't rename or drop columns, can't migrate data, and you never see exactly what it will do (Task 9 bonus). In real projects the schema lives in **versioned SQL files** in Git (V1, V2, V3...). Each file runs once, in order, on every environment. Hibernate only **validates** that the entities match. Tomorrow's AWS database will start empty, so the schema must be reproducible from scratch.

PROBLEM STATEMENT

Before deploying anywhere, schema changes must be reviewed SQL scripts, not something Hibernate guesses at startup.

WHAT YOU NEED TO DO

1. Add the dependencies `org.flywaydb:flyway-core` **and** `org.flywaydb:flyway-database-postgresql` (Flyway 10 needs both).
2. Export the current schema to use as your starting point:

```
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" -U student -h localhost -d studentdb `
   --schema-only -t students -t departments > schema.sql
```

3. Clean it up (remove `SET`, `OWNER` and comment lines) and save it as `src/main/resources/db/migration/V1__baseline.sql`.
4. Your local DB already has these tables, so set `spring.flyway.baseline-on-migrate: true` and `baseline-version: 1`. Flyway then marks V1 as already applied.
5. Set `ddl-auto: validate` and start the app. It should start cleanly.
6. Write `V2__add_student_status.sql`: a column `status varchar(20) not null default 'ACTIVE'`. Add the field to the entity and the response DTO. Restart, then run `SELECT * FROM flyway_schema_history;`.
7. **Prove reproducibility:** as the `postgres` user, run `CREATE DATABASE studentdb_fresh OWNER student;`. Start the app with `$env:DB_URL="jdbc:postgresql://localhost:5432/studentdb_fresh"`. Flyway runs V1 and V2 from scratch and validation passes.
8. Experiment: change one character in V1 and restart. Read the **checksum mismatch** error, then revert the change.

HINTS (TRY WITHOUT THEM FIRST)

If validation fails with a type mismatch, compare the column type in SQL with what Hibernate expects (for example `varchar(255)`, `timestamp(6) with time zone`). Fix it in a **new** migration, never in an applied one.

CHECK YOU'RE DONE

- ☐ `ddl-auto` is `validate`, and the app starts against both the old and the fresh database.
- ☐ `flyway_schema_history` lists your migrations.
- ☐ You saw the checksum error and understand why applied migrations are never edited.

FILES YOU'LL TOUCH

- pom.xml
- application.yml

- db/migration/V1__baseline.sql (new)
- db/migration/V2__add_student_status.sql (new)

THINK ABOUT IT

How would you rename the column `course` to `course_name` with **zero downtime** while old and new app versions are both running? (Look up the expand–contract pattern.)

TASK 22 **LEVEL 4 · ADVANCED** Time: ~90 min

Fix the dual-write problem: the Transactional Outbox

You will learn: dual-write problem, Transactional Outbox pattern, @Scheduled, at-least-once delivery, idempotent consumers

WHY THIS MATTERS

This is the "known limitation" in your technical guide. Saving to PostgreSQL and sending to Kafka are two systems with no shared transaction, so one can succeed while the other fails. The **Outbox pattern** fixes this. The event is written into an `outbox_events` table in the **same DB transaction** as the student. A separate poller later sends unsent rows to Kafka and marks them as sent. If the student is saved, its event **will** be published eventually.

PROBLEM STATEMENT

First prove the problem exists, then fix it so that Kafka downtime never loses or invents events.

WHAT YOU NEED TO DO

Part A: observe the problem

1. Run `docker compose stop kafka` and POST a student. How long does the request take? What status comes back? Is the row in the database? Write down what **actually** happens.
2. Think through the opposite failure: the Kafka send succeeds, then the DB commit fails (the UPDATE from Task 12 is flushed at commit time). That produces a "ghost" event for a change that never happened.

Part B: implement the outbox

1. Write a migration `V3__outbox.sql`: table `outbox_events` (id bigserial PK, aggregate_id bigint, event_type varchar, payload text, created_at timestamptz, published_at timestamptz NULL).
2. In the service, replace `producer.publish(...)` with saving an `OutboxEvent` row **in the same @Transactional method**.
3. Create an `OutboxPublisher` with `@Scheduled(fixedDelay = 2000)` (and `@EnableScheduling`). Load unpublished rows oldest first (max 100). Send each with `kafkaTemplate.send(...).get(5, TimeUnit.SECONDS)` so you wait for Kafka's acknowledgement, then set `publishedAt`. Stop at the first failure to keep the order.
4. Test: with Kafka stopped, POST returns 201 **instantly** and rows pile up with `published_at IS NULL`. Start Kafka: within seconds the rows are published and the consumers log them.
5. Update `StudentServiceTest`: it should now verify the outbox save instead of `producer.publish`.

HINTS (TRY WITHOUT THEM FIRST)

Make the producer fail fast so the poller doesn't hang for 60 s: `spring.kafka.producer.properties.max.block.ms: 5000`.

Repository method: `findTop100ByPublishedAtIsNullOrderByIdAsc()`.

CHECK YOU'RE DONE

- ☐ With Kafka down, POST still returns 201 quickly and nothing is lost.
- ☐ After Kafka comes back, every pending event is published in order.
- ☐ The service no longer talks to Kafka directly.

FILES YOU'LL TOUCH

- db/migration/V3__outbox.sql (new)
- outbox/OutboxEvent.java (new)
- outbox/OutboxRepository.java (new)
- outbox/OutboxPublisher.java (new)
- service/StudentService.java
- StudentServiceTest.java

BONUS CHALLENGE

If the poller sends a message and crashes before it marks the row as published, the event is sent **twice** (at-least-once delivery). Add an `eventId` (UUID) to `StudentEvent` and make the email consumer skip ids it has already processed (an idempotent consumer).

THINK ABOUT IT

With 2 app instances, both pollers pick up the same rows and send duplicates. How does `SELECT ... FOR UPDATE SKIP LOCKED` solve this?

TASK 23 **LEVEL 4 · ADVANCED** Time: ~60 min

Retries and a Dead Letter Topic

You will learn: poison messages, DefaultErrorHandler, backoff, retryable vs non-retryable errors, DeadLetterPublishingRecoverer, Kafka headers

WHY THIS MATTERS

Some messages can never be processed. These are called **poison messages**: broken JSON, for example. Spring's default is to retry and then **skip** the message (you saw this in Task 19), which means the data is silently lost. The production pattern is: retry a few times with a delay for **temporary** errors, send **permanent** errors straight on, and park every failed message in a **Dead Letter Topic (DLT)** so a person can inspect it and replay it.

PROBLEM STATEMENT

Ops says: "Failed messages must never disappear, and one bad message must not block the others."

WHAT YOU NEED TO DO

1. Define a `DefaultErrorHandler` bean built with a `DeadLetterPublishingRecoverer(kafkaTemplate)` and `new FixedBackOff(1000L, 2)` (1 attempt + 2 retries, 1 s apart). Spring Boot plugs a `CommonErrorHandler` bean into its listener factory automatically.
2. Create the topic `student-events.DLT` with the **same number of partitions** as `student-events`. By default the recoverer writes to the same partition number.
3. Make JSON errors non-retryable: `errorHandler.addNotRetryableExceptions(JsonProcessingException.class)`. Let the consumer method declare `throws JsonProcessingException`.
4. Test a permanent error: use the console producer to send `this is not json`. Expect **1** attempt, then the message goes to the DLT.
5. Test a temporary error: make the consumer throw a `RuntimeException` when the name contains "flaky". Expect **3** attempts about 1 s apart, then the DLT.
6. Read the DLT with `--property print.headers=true`. Find the headers `kafka_dlt-exception-message` and `kafka_dlt-original-offset`.

CHECK YOU'RE DONE

- ☐ Bad JSON goes to the DLT after 1 attempt, and the "flaky" message after 3 attempts.
- ☐ Messages behind the bad one are still processed.
- ☐ You can read the original error from the DLT headers.

FILES YOU'LL TOUCH

- kafka/KafkaConfig.java
- kafka/StudentEventConsumer.java

BONUS CHALLENGE

Add a small listener on `student-events.DLT` that logs a WARN line "ALERT: message parked in DLT" with the exception header.

THINK ABOUT IT

While a message is being retried, the messages behind it on the same partition wait. Why does Kafka work that way? Spring's `@RetryableTopic` (non-blocking retries) avoids the wait. What do you give up in exchange?

TASK 24 **LEVEL 4 · ADVANCED** Time: ~90 min

Stage 2: everything in Docker Compose

You will learn: multi-stage Dockerfile, .dockerignore, compose networking by service name, volumes, depends_on + healthchecks, .env secrets

WHY THIS MATTERS

This is **Stage 2** of your roadmap and the bridge to AWS: tomorrow you'll run this exact compose file on an EC2 machine. A **multi-stage** Dockerfile builds the JAR with Maven, then copies only the JAR into a small Java runtime image. Inside Compose, containers find each other **by service name** (`postgres`, `kafka`), not `localhost`.

PROBLEM STATEMENT

One command, `docker compose up -d --build`, must start the app, PostgreSQL and Kafka, fully working, on any machine that has Docker installed.

WHAT YOU NEED TO DO

1. Create a `.dockerignore` that excludes `target/`, `.git/`, `docs/` and `scripts/`.
2. Write a multi-stage `Dockerfile`. **Stage 1** `maven:3.9-eclipse-temurin-17`: copy `pom.xml` first, run `mvn -B dependency:go-offline` (this layer is cached), then copy `src` and run `mvn -B package -DskipTests`. **Stage 2** `eclipse-temurin:17-jre`: copy the JAR, run as a non-root user, `EXPOSE 8080`, `ENTRYPOINT ["java","-jar","/app/app.jar"]`.
3. Add a `postgres` service: image `postgres:16`, `POSTGRES_DB/USER/PASSWORD`, a named volume `pgdata`, ports `5433:5432` (your Windows PostgreSQL already uses 5432), and a `pg_isready` healthcheck.
4. Add an `app` service: `build: .`, ports `8080:8080`, env `DB_URL=jdbc:postgresql://postgres:5432/studentdb`, `KAFKA_BOOTSTRAP_SERVERS=kafka:9092`, and `depends_on` with `condition: service_healthy` for postgres and kafka.
5. Stop your local `mvn spring-boot:run` (it uses port 8080). Run `docker compose up -d --build` and then `docker compose logs -f app`. Watch Flyway create the schema on the empty database.
6. Run your smoke-test script against it. It should pass without changes.
7. Persistence: `docker compose down` then `up -d`: is the data still there? Now `docker compose down -v`: what happened, and why?
8. Secrets: move the passwords into a `.env` file (Compose reads it automatically), keep `.env` out of Git, and commit a `.env.example` instead.

HINTS (TRY WITHOUT THEM FIRST)

Why `kafka:9092` and not `localhost:29092`? Inside a container, `localhost` means **that container itself**. This is the `PLAINTEXT://kafka:9092` listener your compose file already defines for this exact stage.

Open a shell into the DB: `docker compose exec postgres psql -U student -d studentdb`.

Compare image sizes with `docker images`. Why is the JRE base image much smaller than the Maven one?

CHECK YOU'RE DONE

- ☐ One command starts all 3 containers, and `docker compose ps` shows them healthy or running.
- ☐ The smoke test passes against the containerised app, and Kafka events flow.
- ☐ Data survives `down`/`up` and disappears with `down -v`, and you can explain why.
- ☐ No password is committed to Git.

FILES YOU'LL TOUCH

- Dockerfile (new)
- .dockerignore (new)
- docker-compose.yml
- .env (new, not committed)
- .env.example (new)

BONUS CHALLENGE

Add a healthcheck for the app using `/actuator/health` from Task 16. First check whether the runtime image has `curl` or `wget`: `docker compose exec app sh -c "which curl wget"`.

THINK ABOUT IT

Why does the app need `depends_on` with `condition: service_healthy` instead of plain `depends_on`? And why is it fine to skip tests (`-DskipTests`) inside the Docker build?

TASK 25 **LEVEL 4 · ADVANCED** Time: ~90 min

Capstone: end-to-end integration test with Testcontainers

You will learn: @SpringBootTest, Testcontainers, @ServiceConnection, TestRestTemplate, Awaitility for async checks, unit vs integration tests

WHY THIS MATTERS

Your unit tests mock the database and Kafka. An **integration test** proves that the real pieces work together: Flyway migrations, real SQL, JSON, the outbox poller and Kafka. **Testcontainers** starts temporary PostgreSQL and Kafka containers for the test run and throws them away afterwards. This test uses everything you built today.

PROBLEM STATEMENT

Before the AWS deployment, your lead wants one automated test that proves the whole flow: HTTP POST → row in PostgreSQL → outbox → event on Kafka.

WHAT YOU NEED TO DO

1. Add test-scope dependencies: `spring-boot-testcontainers`, `org.testcontainers:junit-jupiter`, `org.testcontainers:postgresql`, `org.testcontainers:kafka`, `org.awaitility:awaitility`. Spring Boot manages their versions.
2. Create `StudentFlowIntegrationTest` with `@SpringBootTest(webEnvironment = RANDOM_PORT)` and `@Testcontainers`. Name it `...Test`, not `...IT`, so that `mvn test` picks it up.
3. Test 1: POST → 201, GET by id returns the same data.
4. Test 2: after a POST, wait up to 10 s (Awaitility) until the outbox row has `publishedAt` set **and** a test `KafkaConsumer` reading `student-events` receives a `STUDENT_CREATED` event with that id.
5. Test 3: POST a duplicate email → 409 in your Task 15 error format.
6. Run `mvn test` with Docker Desktop running. The first run downloads the images and is slow; later runs are faster.

HINTS (TRY WITHOUT THEM FIRST)

```
@Container @ServiceConnection
static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");

@Container @ServiceConnection
static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.6.1"));

@Autowired TestRestTemplate rest;
```

`@ServiceConnection` replaces all the manual datasource and Kafka properties. Spring Boot reads the URLs straight from the containers.

The `KafkaContainer` above is `org.testcontainers.containers.KafkaContainer`, which matches the Testcontainers version that Spring Boot 3.3 brings in. For the test consumer, build a plain `KafkaConsumer<String,String>` with `kafka.getBootstrapServers()`, a unique group id and `auto.offset.reset=earliest`.

```
await().atMost(Duration.ofSeconds(10))
       .untilAsserted(() -> assertThat(outboxRepository.findAll())
                               .allMatch(e -> e.getPublishedAt() != null));
```

CHECK YOU'RE DONE

- ☐ All 3 integration tests pass alongside your unit and web tests.
- ☐ During the run, `docker ps` shows temporary containers, and they are gone afterwards.
- ☐ You can explain when to write a unit test, a web-slice test and an integration test.

FILES YOU'LL TOUCH

- pom.xml
- src/test/java/.../StudentFlowIntegrationTest.java (new)

THINK ABOUT IT

Integration tests are slower but catch more. Picture the "test pyramid": roughly how many of each kind would you want in a project this size, and why?

Graduation checklist

When you can explain each of these in your own words, without notes, you have really learned the local stage:

- ☐ The three layers (controller, service, repository) and why each exists.
- ☐ How a POST travels from HTTP to a database row and a Kafka message.
- ☐ Dirty checking, the persistence context, LAZY vs EAGER loading, and the N+1 problem.
- ☐ Why validation sits both in Java and in the database.
- ☐ DTOs vs entities, and how errors become HTTP responses.
- ☐ Topic, partition, offset, consumer group, lag, key-based ordering, DLT.
- ☐ The dual-write problem and how the Transactional Outbox fixes it.
- ☐ Unit vs web-slice vs integration tests.
- ☐ Configuration precedence, profiles, and keeping secrets out of Git.
- ☐ Multi-stage Docker builds and how containers find each other in Compose.

How today connects to tomorrow (AWS)

Today's task	Tomorrow on AWS
7 · Env vars and no secrets in Git	Configure the app on EC2 with env vars; store secrets in SSM Parameter Store / Secrets Manager
16 · Actuator health	Load balancer / ECS health checks call /actuator/health
21 · Flyway	A brand-new RDS PostgreSQL database gets its schema automatically on first start
24 · Docker Compose	The same compose file runs on an EC2 instance; later the image goes to ECR
25 · Integration tests	The test suite runs in a CI pipeline (GitHub Actions) before every deploy

Good luck, and commit after every task.